ChatGPT

Gordon: A Context-Aware AI Chatbot for Docker Developer Assistance

Abstract

Docker's AI chatbot **Gordon** is an embedded intelligent assistant designed to enhance developer productivity by integrating generative AI directly into Docker Desktop and CLI. This paper provides a comprehensive analysis of Gordon's technical architecture, functionality, and use cases, situating it in the landscape of AI developer assistants. Gordon's design emphasizes **context-awareness** – leveraging local project files, container states, and up-to-date Docker documentation – combined with OpenAI's function-calling capabilities to interpret user queries and perform relevant actions [1] [2]. We detail how Gordon assists with tasks such as Dockerfile optimization, container troubleshooting, and command generation, and how it interfaces seamlessly with Docker's GUI and command-line workflows [3] [4]. The paper compares Gordon to similar AI assistants (e.g., GitHub Copilot Chat, Amazon's Q CLI agent, Anthropic's Claude Code), highlighting common approaches in context integration and tool use, as well as key differences in scope and autonomy [5] [6]. We discuss the implications of Gordon's **function-calling** tool framework, its approach to retrieval-augmented knowledge from Docker's knowledge base [7], and the interaction model that balances automated assistance with user control. Current limitations – including its read-only operation and LLM inaccuracies [8] – are analyzed alongside privacy and security considerations. Finally, we outline future development directions for Gordon, such as enabling automated modifications, expanding contextual inputs, and deeper integration across developer tools [9] [10]. This study illustrates how a domain-specific, context-aware AI assistant can transform developer workflows by providing expert guidance **in situ**, reducing context-switching and improving efficiency in containerized application development.

Introduction

Recent advancements in large language models (LLMs) have led to a new generation of AI assistants for software development. Tools like GitHub Copilot, Amazon CodeWhisperer, and ChatGPT have demonstrated the potential of AI to help write code, debug issues, and answer programming questions. However, these general-purpose assistants often lack direct awareness of a developer's local environment and specific domain context. In complex ecosystems like containerized application development, developers must master a wide range of technologies and continually consult documentation for best practices [11]. Docker, as a leading platform for containers, identified an opportunity to integrate AI assistance *within* its tools to eliminate the friction of switching to external resources [4].

Docker's answer is **Gordon**, an AI-powered chatbot (internally codenamed *Project: Gordon*) that lives inside Docker Desktop and the Docker CLI [4]. Introduced in late 2024 as a beta feature, Gordon is a context-aware agent tailored to Docker-related tasks. Unlike standalone chatbots, Gordon is embedded directly into the developer's workflow: it can be invoked through a chat interface in Docker Desktop or via the `docker ai` command in the terminal [12] [13]. By "meeting developers where they are," Gordon provides help exactly at the point of need – for example, a user can click an **"Ask Gordon"** button next to a failing

container in Docker Desktop to get troubleshooting advice [14] [15]. This tight integration aims to streamline the development experience by providing expert guidance without leaving the Docker environment.

Gordon was conceived to address several pain points that Docker observed with conventional use of AI assistants. First, general LLMs require sufficient context to give accurate answers – "garbage in, garbage out" as the saying goes [16]. Developers asking container-related questions to off-the-shelf chatbots often received suboptimal answers because the models lacked knowledge of the user's specific Docker setup or the latest best practices. Indeed, early experiments with ChatGPT and Claude showed that while they could sometimes optimize a Dockerfile, they frequently introduced subtle bugs or missed modern best practices [17]. The root cause was that the models' training data on Docker topics was often outdated or dominated by poor examples [18]. Second, using an external AI service causes context-switching: a developer must leave their IDE or CLI to consult the assistant, disrupting the flow. Finally, Docker envisioned an assistant that could not only answer questions but eventually perform actions on the user's behalf – something beyond the capabilities of a simple Q&A chatbot [16] [19]. These considerations led to the development of Gordon as an in-product assistant that leverages Docker's own knowledge resources and the user's context to provide more reliable, actionable support.

This paper examines Gordon's architecture and functionality in depth. We describe how Gordon works under the hood – from its client–server design to its use of OpenAI's function-calling API for tool integration. We then explore key use cases and capabilities of Gordon, illustrating how it helps with Docker-centric tasks such as container troubleshooting, Dockerfile improvements, and security checks. In the **Methodology** section, we outline the technical design decisions and the "agentic" workflow that Gordon follows to understand queries and gather context. The **Results** section highlights Gordon's demonstrated capabilities and the early outcomes observed from its deployment. In the **Discussion**, we compare Gordon with other AI developer assistants, analyzing how context-awareness and tool usage differentiate these systems. We also discuss design trade-offs, such as Gordon's focus on providing suggestions with source citations [20] versus more autonomous code-generation approaches. The **Limitations** section addresses the current constraints of Gordon – including accuracy challenges common to LLM-based tools [8] and the intentional restriction against automatic file writes [21] – as well as considerations like privacy and security. Finally, in **Future Work**, we discuss planned enhancements to Gordon and broader implications for AI integration in developer tooling. By providing a thorough analysis of Docker's Gordon assistant, we aim to inform both practitioners and researchers about the potential and challenges of context-aware AI agents in software development workflows.

Methodology (Technical Architecture)

Figure 1: High-level architecture of Docker's Gordon AI assistant. The pipeline involves interpreting the user's query, gathering context via defined tools (e.g., reading local files, retrieving documentation), formulating a prompt for the LLM, and returning an answer or action.

At its core, Gordon employs a hybrid architecture combining local context gathering with cloud-based AI processing. The system is composed of two main parts: a **client-side component** embedded in Docker Desktop/CLI, and a **cloud-based backend service** hosted by Docker [22]. This design allows Gordon to

bridge the developer's local environment with the power of large language models running on Docker's servers. Below, we detail the major steps and components in Gordon's workflow, as illustrated in Figure 1.

1. **User Query Interpretation:** When a developer asks a question or issues a request, Gordon first needs to interpret the intent and decide what actions are needed. This is implemented using OpenAI's *function calling* (tool calling) capability [1]. In practice, Docker has defined a set of **tools/functions** that Gordon can use – for example, a function to retrieve a Dockerfile from the current directory, a function to search the Docker documentation, a function to inspect the status of a container, etc. Gordon's backend LLM (such as GPT-4 via the OpenAI API) is prompted with these tool definitions. The model can choose to output a JSON-formatted function call to invoke a tool if it determines that an external action is required to answer the query [23]. This mechanism lets the AI dynamically decide how to handle different query types. For instance, if the user asks "Why did my container fail to start?", the LLM may invoke a "get_container_logs(container_id)" function to retrieve error logs, whereas a question like "How do I optimize this Dockerfile?" would trigger a function to fetch the Dockerfile contents from the workspace.

2. **Context Gathering:** Once the relevant tool/function is identified, Gordon's client component or backend performs the action to gather context. Because Gordon is integrated with Docker Desktop and CLI, it has privileged access to Docker-specific context:

3. **Local Files and Directories:** Gordon can read files in the user's current project (with user permission). For instance, if the query is about a Dockerfile, the `docker ai` CLI will automatically locate the Dockerfile in the current directory and send its content as context [24]. In Docker Desktop's UI, if the user asks a question related to a file or image, Gordon prompts the user to confirm access to that file/directory [25]. This ensures the assistant can incorporate relevant source code, configuration, or error message data. Gordon **understands the local environment**, including source code and images, enabling personalized guidance that reflects the user's actual setup [2].
4. **Docker Objects (Containers/Images):** Because it hooks into Docker Desktop, Gordon can see information about local containers and images. For example, it knows what containers are running or stopped and can obtain their names, statuses, and logs. If a container crashes, Gordon is aware of that event via the Docker API integration. This context allows it to give specific advice (e.g., suggesting environment variables if a container exited due to a missing password, as in the PostgreSQL example) [15] [26].
5. **Docker Documentation and Knowledge Base:** To supplement the above, Gordon uses a Retrieval-Augmented Generation approach for Docker's extensive documentation. Docker integrated a service called *Kapa.ai* which indexes official Docker docs and can retrieve relevant passages to ground the LLM's answers [27] [7]. When the user's query is conceptual ("How can I run MongoDB in Docker?") or about best practices, Gordon will query this documentation knowledge base to pull the latest authoritative guidance. This design mitigates the issue of LLM training data being outdated or containing incorrect Docker info, by always referencing up-to-date docs content [7]. The retrieved snippets (with source URLs) are included in the prompt to the LLM so that Gordon's answer can incorporate and cite them.

6. **Live Data (Docker Hub, etc.):** Gordon can also fetch live data from Docker Hub and other tools. For example, when recommending a `docker run` command, it might query Docker Hub for the image's details (available tags, default configs) to tailor the command [28]. The Beta launch notes describe that the suggestion for running a container "combines a custom prompt, live data from

Docker Hub, Docker container expertise and private usage insights" unique to Docker [29]. This implies Gordon might use internal analytics (e.g., common usage patterns) as part of its context for recommendations, although the specifics are proprietary.

7. **LLM Prompt Construction:** After gathering the necessary context, Gordon's backend composes a prompt for the LLM. This prompt typically includes:

8. A concise restatement of the user's query or task intent (possibly refined by the function-calling step).
9. Relevant context data: e.g., the content of the Dockerfile or error log (if the query is about those), or excerpts from documentation and knowledge base articles.
10. Instructions for the LLM to produce a helpful answer. Docker likely provides the model with a role prompt that encourages it to be accurate, concise, and include references. Indeed, the design explicitly avoids overly verbose outputs; the team noted they **"intentionally designed the output to be concise and actionable"**, focusing on clear steps or commands rather than long-winded explanations [20]. The model is also instructed to provide source citations for factual recommendations, typically pointing to Docker's docs [20]. This not only improves user trust (they can verify suggestions) but also aligns with an academic style of answer, which helps distinguish authoritative info from possible hallucinations.

11. Tools' results: If a function was called (like retrieving a file), the result of that function (e.g. file content snippet, container status) is appended to the prompt so the model can use it to formulate the answer.

12. **LLM Response and Tool Feedback Loop:** The LLM (hosted on Docker's servers via OpenAI API) processes the prompt and generates a response. In many cases, this will be a direct answer to the user's question, which is then returned to the client to display in the UI or terminal. However, Gordon can also engage in a **feedback loop** using function calling: the model might determine that additional information is needed and invoke another tool. For example, if the user asks "Is my Docker image secure?", the agent might first run a vulnerability scanning tool (perhaps using Docker Scout context [30]) and then, based on the findings, formulate advice. Each function call returns data that the model can incorporate on the next prompt iteration. This iterative tool use follows the intelligent agent paradigm seen in other AI systems where the model can plan a sequence of actions (reminiscent of the ReAct framework or "agentic" behavior) [31]. Docker's team experimented extensively with this tool-calling approach, noting that careful design of tool schemas and descriptions was crucial for reliability [32]. For instance, they found that more detailed tool descriptions with examples improved the model's ability to invoke them correctly [33]. They also observed that different LLMs have varying preferences; the tuning of prompts and tool definitions had to be adjusted depending on whether they used GPT-4 versus GPT-3.5, etc., as the models might otherwise get confused or fail to trigger the intended function [34]. This insight aligns with general observations from OpenAI that newer model versions show improved function call detection [35] [36].

13. **Action Execution and Response Delivery:** In the current beta, Gordon operates in a **read-only, suggestive mode** – it does *not* automatically execute changes on the user's system. Once the LLM finalizes an answer (after any tool calls), that answer is delivered back to the user through the interface. If the answer includes a recommended command (say a corrected `docker run` invocation or an optimized Dockerfile snippet), it is up to the user to apply it. For example, Gordon

might respond: *"Your container failed because POSTGRES_PASSWORD was not set. You can fix it by running:* `docker run -e POSTGRES_PASSWORD=<secret> postgres`*"* [26]. The user can then copy-paste or directly run this suggestion. In Docker Desktop's UI, Gordon's answers appear in a chat panel (with formatting for code or commands as needed), and in the CLI it prints to the console. The design choice to not have Gordon automatically make changes was intentional for the beta phase, due to safety: *"Currently, Gordon does not have write access to any files, so it will not edit your files"* [21]. All actions that affect the system are left under user control. This conservative approach is contrasted with some other emerging tools (discussed later) that do attempt autonomous code modifications [37] [38]. In Gordon's case, any automation (like creating a Dockerfile or adjusting settings) is done through suggestions or via Docker Desktop's guided UI flows rather than silent background actions.

Throughout this process, **privacy and security** measures are in place. When Gordon sends context data to the backend LLM, Docker ensures it is transmitted securely (encrypted in transit) and not stored permanently [39]. The data (files, logs, etc.) is used only to formulate the answer and then discarded, per Docker's documentation. Additionally, Docker anonymizes usage telemetry: queries and responses can be logged for improving the service, but they are not tied to user identities and are not used to train the base AI model [40] [41]. Users must explicitly opt-in to enable Gordon, agreeing to these terms, and they retain the ability to disable the assistant at any time [42] [43]. These considerations are important in enterprise settings where sensitive code might be present; Docker's approach aligns with a trend of on-prem or controlled-context AI assistants that differ from public AI chatbots in privacy guarantees.

In summary, Gordon's methodology combines *context retrieval*, *function/tool execution*, and *LLM reasoning* in a closed loop. This architecture allows it to be highly specialized: it acts as a Docker expert that always consults the right references (like an internal Stack Overflow of Docker knowledge plus awareness of your project). By offloading the heavy AI computations to the cloud, Docker can leverage state-of-the-art models (such as GPT-4) without requiring end-users to have any special hardware. Meanwhile, the local integration ensures the answers are tailored and immediately applicable to the user's situation. In the next section, we examine what Gordon can do with this setup – its functionality and some results observed so far.

Results (Functionality and Use Cases)

Gordon's effectiveness can be seen in the range of Docker-related tasks it supports. Docker's team has reported that even in its early beta, Gordon is "already really good at some things" [44]. Here we outline key use cases and capabilities demonstrated by Gordon, along with examples and initial user outcomes:

- **General Docker Q&A:** Gordon serves as an on-demand Docker expert. It can answer questions about Docker concepts, commands, and best practices with a high level of accuracy, because it draws on the official documentation and other curated sources [3]. For instance, a user might ask, "What's the difference between a bind mount and a volume?" or "How do I expose a port in my Dockerfile?". Gordon will provide an explanation or step-by-step guidance, often with references to the docs. This is similar to the role of Docker's documentation website or forums, but made more interactive and context-aware. Unlike a generic AI chatbot, Gordon is less likely to hallucinate irrelevant answers in this domain, as it confines its knowledge to Docker's verified information (via the Kapa.ai RAG integration) [7]. The result is instant, accurate answers delivered right in the development environment, which saves developers the effort of manually searching through docs or Google.

- **Troubleshooting Containers:** One of Gordon's most compelling use cases is diagnosing errors in container operations. When a container fails to build or run, developers traditionally have to interpret error messages and search for fixes. Gordon automates this process. For example, consider a container that exits immediately due to a missing environment variable (such as the PostgreSQL password error) – Docker Desktop will show the container in a "stopped" state with a cryptic log message. With Gordon enabled, the developer can click the icon next to that container or open the "Ask Gordon" tab for it [15] [26]. Gordon will analyze the container's logs and configuration and explain the cause of the error, then suggest a solution. In the PostgreSQL case, Gordon explains that a superuser password is required and proposes the correct `docker run -e POSTGRES_PASSWORD=... postgres` command to resolve it [26]. This kind of targeted support significantly reduces debugging time. Early usage indicates that Gordon is adept at recognizing common failure patterns (e.g., port conflicts, missing environment variables, insufficient memory allocation) and providing the specific fix or a pointer to relevant documentation (like a link to a Docker Hub image's usage instructions). By surfacing suggestions contextually (a UI button appears only when there's an error to troubleshoot), it ensures the help is timely and relevant [45]. This contextual troubleshooting exemplifies how Gordon's integration into Docker Desktop adds value beyond what a standalone chat window could do.

- **Dockerfile Analysis and Optimization:** Gordon can act as a smart reviewer for Dockerfiles and Docker Compose files. Developers can ask Gordon to "rate my Dockerfile" or "improve my Dockerfile," either via the CLI in the project directory or through Docker Desktop's interface [46]. Gordon will parse the Dockerfile and provide feedback across multiple dimensions: security best practices, image size efficiency, build cache utilization, maintainability, and more [47]. For example, if a Dockerfile is missing `HEALTHCHECK` or is installing unnecessary packages, Gordon might point that out. It might suggest multi-stage builds to reduce image size, or recommend using official base images that are more secure. The result is a list of actionable suggestions to make the Dockerfile follow Docker's published best practices (Docker had documented such best practices, and those were part of the data provided to the model) [48]. In one anecdote shared by Docker, when analyzing a Node.js project, Gordon recommended using Node v20.12 – not the absolute latest version, but the one matching the version found in the project's *package.json* file [49]. This shows that Gordon contextualizes its advice to the application at hand, rather than giving generic suggestions. Such tailored recommendations help developers modernize and optimize their container configurations with confidence. While a human expert could do the same code review given enough time, Gordon provides it instantly. Users have found this especially useful for learning; by seeing Gordon's explanation of *why* a certain change is beneficial, they gain insight into Docker best practices.

- **Guidance on Running Containers and Images:** For users exploring new images or setting up services, Gordon can suggest the proper `docker run` or `docker compose` usage. The assistant can incorporate real-time data from Docker Hub about an image's default settings or popular configurations [28]. As a result, a query like "How do I run a MongoDB container with persistence?" might yield a concise answer with the exact command (including `-v` for volume mount and necessary `-e` variables) along with a brief explanation. In Docker Desktop, when you click "Run" on an image, the UI even provides a hint like *"Unsure about parameters? Ask Gordon"* with a suggested question [50]. This lowers the barrier for developers to deploy containers correctly. Preliminary user feedback indicates that Gordon's suggestions often save time by eliminating trial-and-error. Instead of searching Docker Hub docs and assembling a command from examples, the developer gets a one-shot recommended command. Moreover, Gordon typically cites the source (for instance, linking

to Docker's documentation for that image or command) [20], which adds confidence that the recommendation is vetted.

- **Integration with Docker Scout (Security/Policy):** Gordon also helps with security and configuration compliance issues. Docker Scout is a service that flags vulnerabilities and policy violations in images. If Scout identifies, say, an outdated base image or an insecure package in an image, Gordon can suggest remediation steps [30]. For example, if an image is using Ubuntu 18.04 which is past end-of-life, Gordon might suggest upgrading to a later base image and provide a Dockerfile snippet to do so. Or if a company policy says images must not run as root, and Scout flags this, Gordon could explain how to add a non-root user in the Dockerfile. These capabilities make Gordon a helpful assistant for DevSecOps, turning static analysis results into practical advice. While detailed results of this integration have not been fully published, the Docker team explicitly lists "remediating policy deviations from Docker Scout" as something Gordon can do [30], indicating its utility in security contexts.

- **DevOps and CI/CD Assistance:** Although primarily focused on local development, Gordon's knowledge extends to Docker-related workflows, including GitHub Actions and other CI configurations [51]. A user can ask, for instance, "How do I set up a GitHub Actions workflow to build and push a Docker image?". Gordon can provide a template or pointers, since it has been fed documentation covering Docker's GitHub Action and general CI best practices. This shows the assistant's versatility: it is not limited to the Docker engine itself, but covers the surrounding ecosystem (Compose files, Kubernetes manifests, etc., to some extent). Essentially, anything documented under the Docker tool suite (CLI, Desktop, Hub, Compose, Docker's Kubernetes integration, etc.) is fair game for Gordon's Q&A [52]. Early user interactions likely span a broad array of such topics; while we do not have quantitative data on which queries are most common, Docker did share that the documentation AI (a precursor to Gordon) was handling about 1,000 queries per day shortly after launch [53]. This suggests significant interest and usage, which presumably carries over as Gordon's beta access expands.

Figure 2: Example of Gordon's CLI output (from Docker's beta preview). Here, the user asked the Docker AI for help in a Node.js project; Gordon recommended a command using Node 20.12, matching the version found in the project's files [49]. The assistant provides a concise explanation along with the command.

Collectively, these results demonstrate Gordon's promise in improving developer efficiency and knowledge. Qualitatively, the feedback has been positive – Docker described Gordon as delivering "actionable suggestions and automations that remove many of the manual tasks" in container development [54]. By saving developers from context-switching and by providing expert advice on demand, Gordon can shorten the debugging cycle and learning curve. In essence, Gordon turns the Docker documentation and years of community knowledge into an interactive mentor available 24/7. This is particularly beneficial for newcomers (who can use it to learn correct Docker usage in real time) and for experienced users (who can get quick answers or confirmation without breaking flow).

It's worth noting that Gordon's answers often include hyperlinks to documentation pages or other reference material [20]. This practice not only lends credibility but also helps users delve deeper if needed. In an academic sense, Gordon is providing citations for its claims – a critical feature when trusting an AI in a professional toolchain. The ability to trace an answer back to the official docs addresses one of the main

criticisms of AI assistants (that they are "black boxes"). Docker's choice to incorporate sources is a result in itself: it showcases a best practice in AI assistant design that others might follow to increase transparency.

While quantitative evaluations (e.g., measuring time saved or error reduction) have not been published yet, the variety of use cases handled successfully by Gordon indicates a strong alignment with developer needs. As the beta continues, we expect more empirical data to emerge on Gordon's impact (such as user satisfaction scores or performance on standardized troubleshooting tasks). For now, the evidence from Docker's internal testing and user anecdotes suggests that Gordon achieves its primary goal of being a "helpful, expert-level guide on Docker-related questions and technologies" [54] [55]. In the next section, we place Gordon in context with other AI assistants and discuss some of the design considerations and trade-offs illuminated by these results.

Discussion

Comparison with Other AI Developer Assistants: Gordon is part of a broader wave of AI copilots entering developer tools, yet it has distinctive features due to its domain focus and integration level. A comparison with a few contemporaries helps illustrate common trends and unique aspects:

- *GitHub Copilot Chat (Editor-based Assistant):* GitHub's Copilot started with code autocomplete and has evolved to an IDE chat assistant (Copilot Chat) that can answer questions about your code and suggest edits. Like Gordon, Copilot Chat is context-aware in that it reads the user's current code buffer or project files to tailor its responses. However, Copilot's primary domain is code (multiple programming languages), whereas Gordon's domain is the Docker/container ecosystem. One key difference is the type of actions: Copilot can suggest code changes or even auto-edit a file in the IDE if permitted, but it doesn't directly execute shell commands or integrate with external tools out-of-the-box. Gordon, on the other hand, focuses on Docker commands and configurations, and is integrated with the Docker runtime environment. This means Gordon can provide advice that spans beyond code into deployment and environment setup (e.g., launching containers, configuring images), which typical coding assistants might not handle well. Moreover, Gordon's function-calling approach gives it a structured way to use tools (like reading a Dockerfile or retrieving logs), something Copilot's more free-form model lacks. In summary, Copilot Chat is a general coding assistant, whereas Gordon is a specialized DevOps assistant – each strong in their arena. A developer might use Copilot to write application code and tests, then turn to Gordon when containerizing the app or diagnosing container issues. We anticipate future integrations (the Docker team even hinted at "enhanced GitHub Copilot integrations" on their roadmap [10]) that could allow these tools to complement each other seamlessly.

- *Amazon Q Developer CLI*: **Announced in late 2023 and improved in 2024, Amazon's Q Developer is an AI assistant targeting software development with deep ties into AWS services. Its** CLI agent **component is perhaps the closest analog to Gordon in terms of operating context. The Amazon Q CLI assistant enables natural language interaction in the terminal and can perform a range of actions – from code generation to running build commands – within an AWS-focused development environment** [56] [57]**. One major difference is that Amazon Q's agent has progressed to** autonomously executing commands **on the user's behalf when appropriate, thanks to its integration with tools like compilers, package managers, and the AWS CLI itself** [5]**. In a demonstration, it was shown to scaffold a complete project (creating files, initializing a git repo, installing dependencies) without direct user intervention for each step** [37]**. In**

contrast, Docker's Gordon currently stops short of automatic execution – it suggests actions but leaves the actual command running to the user [21]. This reflects a more cautious approach, likely to prevent unintended changes and maintain user control. Another aspect is the scope of domain: Amazon Q is aimed at general software projects (especially leveraging AWS cloud resources), whereas Gordon is tightly scoped to container-related tasks in the Docker ecosystem. Both systems highly value context: Amazon Q can "understand your codebase" and even read/write local files to achieve goals [37], and it employs step-by-step reasoning for complex tasks [5]. Gordon uses context in a more advisory capacity, pulling in information but not yet making persistent changes. Additionally, Amazon Q's assistant draws on Amazon Bedrock *and large models like Anthropic's Claude under the hood [57], indicating significant computational heft. Gordon's backend uses OpenAI's models and Docker's own knowledge base. Both approaches underscore that integrating reliable knowledge sources* is key: Amazon Q excels with AWS-specific tasks by having tight integrations with AWS services [6], while Gordon excels with Docker by leveraging Docker's internal docs and context. In essence, each assistant is vertically integrated into its platform's stack – a trend we expect to continue (e.g., domain-specific AI copilots for specific frameworks or cloud platforms).

- *Anthropic Claude Code*: **Still in research preview (as of 2025), Claude Code is another AI coding assistant that distinguishes itself by** autonomously modifying code across an entire project **when instructed [38]. It can analyze a repository, understand the code structure, and apply multi-file edits to implement a high-level request (like adding a feature) [38]. This level of autonomy goes beyond Gordon's current capabilities. However, the problem space is different: Claude Code is aimed at software maintenance and feature implementation in code, whereas Gordon's focus is operations and configuration in the Docker realm. If we view Claude Code as pushing the envelope on *how much authority to give the AI* in a dev environment, Gordon currently sits on the more conservative end of that spectrum. Docker has explicitly stated that they are working on features to allow Gordon to "do the work for you, instead of only suggesting solutions" in the future [9]. This suggests that Gordon may gradually move towards controlled automation (perhaps first by writing back Dockerfiles or Compose files with user approval). The comparison highlights an important design consideration:** trust and safety vs. automation**. While tools like Claude Code and Amazon Q demonstrate that it's technically feasible for an AI to modify code or run commands autonomously, Docker's incremental approach indicates a priority on earning user trust and ensuring correctness before unleashing full automation. Given the potential impact on infrastructure (improper commands could delete containers or degrade security), this caution is warranted. In practice, the best path may be** mixed-initiative** interaction: Gordon could propose changes and even prepare them, but ask the user for confirmation to apply them – a balance between efficiency and oversight, which is something also seen in Claude's design (Claude Code requests permission before executing risky operations) [58].

- *Other Domain-Specific Assistants:* Gordon is not alone in the idea of embedding AI in developer tools for specific domains. For example, JFrog (an artifact management platform) introduced an AI assistant to translate natural language into JFrog CLI commands for DevOps tasks [59]. JetBrains has integrated an AI assistant across its IDEs that provides context-aware code generation and explanations within those environments [60]. These tools, like Gordon, aim to reduce the cognitive load by letting developers use natural language to accomplish tasks that usually require memorizing syntax or searching docs. A common thread is context-awareness: JetBrains' AI knows about your code project, JFrog's knows about Artifactory commands, and Gordon knows about your Docker

context. This reinforces that the next generation of AI assistants are **embedded** – they leverage local and domain-specific context to go beyond what a generic chatbot can do. It's a trend from generic to specialized: while one can ask ChatGPT general Docker questions, Gordon's advantage is being *on the ground* in the Docker CLI with direct access to your environment, leading to more precise and actionable help.

Context-Awareness and Reliability: A significant observation from Gordon's development is the importance of providing the LLM with high-quality, task-specific context. Docker's initial trial with vanilla LLMs (ChatGPT/Claude without extra context) resulted in unreliable answers for Docker problems [17]. By injecting context (local state, files, logs, plus current documentation), Gordon dramatically improves the relevance and accuracy of responses. This exemplifies the broader AI principle of *Retrieval-Augmented Generation (RAG)*, where an information retrieval step supplies up-to-date knowledge to the model [7]. The advantage is twofold: it mitigates the model's knowledge cutoff or biases in training data, and it allows customization of the assistant's expertise (in this case, making it a Docker expert). The success of Docker's documentation assistant (which served millions of doc page views) [61] [7] likely paved the way for Gordon's approach. In academic terms, one could say Gordon's architecture treats the LLM as a reasoning engine that must be properly "prompt-conditioned" with situational context to produce valid results – reflecting findings in current research that grounding LLMs with context greatly enhances their utility in specialized domains.

However, context-awareness comes with challenges. One is **context length limits**: LLMs have finite prompt sizes. If a user's Dockerfile is very large or if there are multiple relevant files (Dockerfile, docker-compose.yaml, .env, etc.), Gordon must be judicious in what to include. The system likely uses strategies such as summarizing or focusing on salient parts of files. Another challenge is ensuring the context is correctly interpreted by the model. If logs are noisy or a Dockerfile has unusual syntax, the model might mis-read them without additional guidance. Docker's use of function calling helps here – by structuring the input (e.g., providing the file content with a function name that implies what it is), it gives the model clearer signals.

Moreover, context usage raises privacy considerations: sending code or configurations to a cloud service can be sensitive. Docker's decision to make Gordon opt-in and to be transparent about data handling [39] [40] is crucial for user acceptance, especially in corporate environments. In the future, one could envision an on-premises version or a mode of Gordon that uses a local LLM (possibly via Docker's Model Runner) for companies that cannot send data outside. The present design leans on Docker's servers and OpenAI, which is convenient but may not satisfy all users' privacy needs.

Function-Calling and Tool Use: Docker's experience with OpenAI's function-calling reflects a microcosm of the state of the art in connecting LLMs with external tools. They found that writing good tool definitions and testing extensively are necessary to avoid misfires [33]. An interesting point is that the choice of model affects tool usage; e.g., one model might over-eagerly call a tool, while another might underuse it, given the same prompt [62]. This suggests that there is still an art to engineering these AI agent systems – the behavior is not yet fully predictable. Docker's iterative experimentation (trying different prompt phrasings, adding/removing tools to see impact [33]) is akin to what early research on *interactive LLM agents* has reported: slight differences can lead to very different chains of actions.

The design also had to consider failure modes: what if a tool call returns something unexpected or if the LLM hallucinated a function name that doesn't exist? In Gordon's case, the set of tools is probably

hardcoded and the system can catch invalid calls, but robust error handling would be an important engineering task. As this technology matures, we expect best practices to emerge (OpenAI's own documentation on function calling is evolving as developers share tips). Gordon's development highlights the need for clear tool semantics for the model and for possibly restricting the model's outputs to valid calls – techniques like inserting a "function planner" layer or using few-shot examples of tool use can help an AI agent stay on track [33].

Another aspect is user-defined actions. Currently, users cannot extend Gordon with new tools; it comes with a fixed set geared toward Docker. In the future, perhaps Docker or third parties might let users plug in custom function calls (for example, a tool to query an internal company knowledge base, or to interface with a particular CI system). This modularity would increase Gordon's power but also complexity. It parallels ideas in other AI assistant frameworks where an agent can have a suite of plugins. Indeed, one might liken Gordon to a domain-specific instance of the more general concept behind OpenAI Plugins or Microsoft's "Copilot stack", but focused on containers. This specialization likely made implementation easier and results more predictable.

User Experience and Integration: From a human-computer interaction perspective, Gordon's integration strategy is noteworthy. Rather than forcing users into a chat for everything, it augments existing UI elements (with the icons and an extra "Ask Gordon" tab) [45] [63]. This means the assistant is present but not obtrusive; it's invoked contextually. Users still control when to ask Gordon, and the interface provides the affordances at relevant moments (like when something goes wrong, or when a user might be about to configure a container). This design likely improves adoption, as users don't have to remember a separate workflow – the help is naturally available in situ. It echoes a principle found in studies of proactive assistance: timing and relevance of suggestions are key to user satisfaction [64] [65]. If Gordon were simply a generic chat window, users might ignore it or forget it's there. By weaving it into the normal Docker UI/UX, Docker increases the chances that Gordon will actually be used and useful.

One can draw parallels to IDEs showing linting tips or code suggestions at just the right moment, except here it's on a higher level (dev environment instead of code). The feedback so far (as seen in community blog posts and Docker's announcements) is that this seamless integration is a strength [66] [67]. It reduces the mental barrier to using AI help – you ask a question as naturally as clicking a button when you're stuck. For CLI enthusiasts, having `docker ai` means one can stay in terminal and still get AI assistance, which is valuable since many DevOps professionals prefer command-line operations.

Limitations in Practice: Despite its promising capabilities, Gordon inherits certain limitations common to LLM-driven assistants. It sometimes may produce incorrect or suboptimal suggestions; Docker's documentation explicitly warns that as with any LLM-based tool, *"responses may sometimes be inaccurate. Always verify the information provided."* [8]. There is an implicit understanding that Gordon is not infallible – it might misinterpret a question or fail to solve an unusual problem. For example, if a container issue stems from an obscure application bug rather than a Docker misconfiguration, Gordon might not pinpoint it. It is tuned for Docker context, so it might not have expertise in, say, database internals or networking beyond what's in docs. Users must use judgment and treat it as an assistant, not an oracle.

Another limitation is that in the beta, Gordon handles mostly one-shot Q&A style interactions (possibly with follow-up questions). It's not yet a fully conversational agent that maintains a lengthy dialogue or complex goals over time (though one can ask sequential questions in the chat, the context window size will constrain how much history is remembered). This is fine for the typical use cases, but it means Gordon is not (at least

yet) a *planning agent* that could carry out a multi-step task end-to-end without guidance. By comparison, the Amazon Q CLI agent showing multi-step autonomous behavior is a step closer to that ideal [5] [37]. Docker may choose to gradually increase Gordon's autonomy but likely will do so for well-scoped tasks and under user supervision.

We also note that Gordon currently requires Docker Desktop (versions 4.38+). This means it's not natively available on Linux servers where Docker Engine runs without Desktop. Docker Desktop is mostly used on developer workstations (Windows, macOS) and includes the convenience of the GUI and integrated CLI. Therefore, a backend developer working on a Linux machine without Desktop wouldn't have access to Gordon unless Docker provides a standalone CLI plugin in the future. This is more of a product distribution limitation than a technical one, but it affects who can use the assistant. In enterprises, many CI systems or cloud dev environments might not have Docker Desktop, so Gordon's usage may be focused on development time rather than CI/CD automation (whereas something like Amazon Q might be usable in headless environments).

Finally, the effectiveness of Gordon's suggestions is bounded by the quality of Docker's own knowledge base and rules encoded. If best practices evolve (e.g., a new Docker feature changes how we do things), the system needs updating. Docker's use of its documentation via RAG is a smart way to keep answers current, but it will rely on Docker continuously maintaining those docs and possibly fine-tuning prompts as new versions of Docker introduce changes. In other words, Gordon's intelligence is as good as the combination of the LLM plus Docker's curated data – it doesn't learn from the user's own experiences or adapt beyond what's given. Future AI assistants might incorporate learning from user feedback more directly (for instance, adjusting suggestions if users frequently mark them as unhelpful), but in the current form Docker only uses feedback to improve the product offline, not realtime adaptation [40] [41].

In conclusion, the discussion highlights that Gordon exemplifies many of the emerging patterns in AI assistant design: deep integration into a specific domain, use of retrieval and tools to ground the LLM, careful consideration of user experience, and a cautious approach to autonomy. Its development and comparison with others provide insight into how AI can be practically and safely applied to developer tooling. Next, we look at the road ahead for Gordon and similar systems.

Limitations

While Gordon represents a significant step forward in AI-assisted developer tools, it is important to recognize its current limitations. Some of these have been touched on earlier, but we consolidate and elaborate them here in an academic context:

- **Accuracy and Hallucinations:** Like all large language model applications, Gordon can sometimes produce incorrect answers or **hallucinate** – that is, assert something confidently that is not true. For example, if asked an obscure question not covered in documentation, it might generate a plausible-sounding but inaccurate response. Docker's team acknowledges this limitation, noting that Gordon's LLM-based replies "may sometimes be inaccurate" and advising users to verify its suggestions [8]. In the controlled Docker domain with RAG support, gross hallucinations are reduced, but subtle errors can occur. A known issue with code-oriented LLMs is that they might propose commands that don't exactly fit the scenario or are syntactically wrong in edge cases. Gordon attempts to mitigate this by providing sources (so users can cross-check) [20], but ultimately the onus is on the developer to review the advice. This limitation is not unique to Gordon – it is a fundamental challenge for any AI

assistant. Continued improvements in model training (especially domain-specific fine-tuning) and better prompt engineering may improve accuracy over time. Additionally, user feedback loops (thumbs-up/down on answers) are intended to help Docker refine Gordon's performance [40].

- **No Autonomous Actions (Yet):** Currently, Gordon does not make any changes to the user's system on its own. It doesn't write to files, execute commands, or alter configurations directly [21]. While this was an intentional design choice for the beta (to prioritize safety and trust), it is a limitation in terms of automation. Competing solutions, like Amazon's CLI agent or Claude Code, have showcased the productivity gains from letting the AI actually carry out repetitive tasks (e.g., scaffolding projects, applying code changes) [37] [38]. With Gordon, if it suggests 5 changes to your Dockerfile, you must manually edit the Dockerfile accordingly; if it gives a `docker run` command, you must execute it yourself. This interactive friction could be an area for improvement. On the flip side, some users might see this "look but don't touch" mode as a feature, since it guarantees the AI won't do anything behind their back. In any case, until Gordon gains the ability to automate actions (with user permission), it remains more of an advisor than an agent. This limits its utility in scenarios where large-scale or repetitive modifications are needed – the user would have to script those or wait for future enhancements. Docker's roadmap indicates they are working on enabling the agent to perform actions "on the user's behalf" in upcoming versions [9], which should address this limitation gradually.

- **Domain Scope Constraints:** Gordon is expert in Docker and container-related matters, but it is not a general-purpose programming assistant. If a question strays outside the Docker ecosystem (for instance, "How do I optimize my Python code?" or "What does this error in my Django app mean?"), Gordon may not be very helpful. It might attempt an answer based on whatever general knowledge the underlying model has, but that's not its intended use. In such cases, a developer might still need to consult other AI tools or documentation. The **narrow scope** is a double-edged sword: it makes Gordon very effective for what it's designed for, but it means it can't replace all other assistants. In practical terms, this isn't a huge drawback since Gordon is meant to complement, not replace, coding assistants. Yet it's worth noting that the user has to be mindful of what types of queries they ask Gordon. The Docker branding in the name "Ask Gordon" helps set the expectation that it's for Docker questions. Over time, if Docker expands Gordon's knowledge base (perhaps to cover more DevOps topics, Kubernetes, etc.), this scope might widen, but at present it stays in its lane.

- **Requirement of Docker Ecosystem & Latest Version:** To use Gordon, one must be running a modern version of Docker Desktop (v4.38 or later) [68], and be logged in with a Docker account to enable the feature [69]. This poses a limitation for some users:

- Developers on unsupported environments (e.g., pure Linux without Desktop, older Docker versions, or air-gapped systems without internet access) cannot use Gordon as of now. If your development workflow doesn't include Docker Desktop, Gordon is effectively inaccessible.
- The need for a Docker account and opting into a beta program might discourage some users who are wary of telemetry or cloud services. Enterprise admins have to explicitly allow Gordon for their organizations [70], which could be an adoption barrier in companies with strict policies.
- There might also be performance considerations: using Gordon means each query goes to Docker's servers and OpenAI. If either the user's internet or Docker's service is slow, the responses could be sluggish. Heavy usage might also be rate-limited (though no details were given, we can infer Docker

would have some limits to prevent abuse). In an offline scenario, Gordon is completely unavailable, unlike local tools that can be used without connectivity.

- Additionally, because it's in beta, there could be bugs or instability. Docker Desktop betas have occasionally introduced issues (for example, community forums mentioned problems with 4.38.0's initial release, though not necessarily due to Gordon) [71]. Relying on a beta feature for critical work might be risky until it matures.

- **Context and Knowledge Limitations:** While Gordon uses documentation and local context, it may still run into context length limits or knowledge blind spots. If a user asks a very broad question like "Explain everything about Docker networking," the assistant might not be able to provide a full treatise in one answer – it will summarize and possibly miss details. Or if logs are extremely long, Gordon might truncate what it sends to the model. In edge cases, the assistant might say it doesn't know or direct the user to external sources if the query is out of scope. Furthermore, if Docker's documentation has gaps or hasn't documented a new feature yet, Gordon's answers could be incomplete or slightly outdated (until the docs catch up). This is essentially the documentation dependency issue: Gordon is only as good as the available reference material plus the model's base training. For cutting-edge Docker features (say something introduced yesterday in a new release), the assistant might not have integrated that knowledge immediately.

- **Security and Ethical Considerations:** There is an implicit trust users place in Gordon when they allow it to send code or environment info to Docker's servers. Although Docker promises not to store or misuse that data [39], users dealing with highly sensitive code might still be uncomfortable. Additionally, while unlikely, if someone were to prompt Gordon in a way that reveals secrets (imagine asking it to summarize a config file that contains credentials), there is a risk of exposing those secrets to the AI system. Docker's terms likely advise against sending sensitive data, and the user should use discretion. This limitation is more about the *safe use* of the tool rather than the tool itself, but it's an important caveat. On the ethical side, using AI suggestions blindly can lead to problems – e.g., if Gordon suggests a command that accidentally deletes data because the user asked a wrong question and it misunderstood. Thus, a limitation is that the human operator must remain in the loop and cannot entirely rely on Gordon without due diligence. This is a limitation of current AI assistants in general – they augment human work but don't eliminate the need for human judgment.

In summary, Gordon's limitations underscore that it is an evolving helper, not a magical solution. Recognizing these constraints helps set the correct expectations and guides the focus for future improvements. It also highlights why Docker has kept the feature in beta: there's room to refine accuracy, broaden applicability, and ensure robust safety before it can be considered a production-ready co-pilot for all Docker users.

Future Work

Docker's vision for Gordon extends well beyond its initial beta capabilities. Based on official communications and the trajectory of similar AI systems, several key areas of future development and research can be anticipated for Gordon:

- **Enabling Autonomous Actions:** A primary goal is to move Gordon from a read-only advisor to a true *agent* that can execute tasks. Docker has indicated that they are "hard at work on future

features that will allow the agent to do the work for you" [9]. In practical terms, this means adding functionality for Gordon to safely modify Dockerfiles, adjust docker-compose configurations, or even run Docker CLI commands on behalf of the user. Implementing this will require a careful approach: likely an opt-in for each action, where Gordon might present a diff of a proposed change and ask the user to confirm (to avoid any destructive operations). Research on human-in-the-loop AI suggests this mixed-initiative approach can yield efficiency gains while maintaining user trust [58]. We expect to see features like a "Apply Fix" button next to Gordon's suggestions in the UI, or a CLI flag that lets `docker ai` apply certain recommended changes automatically. Over time, if reliability is proven, some of these fixes might even become one-click automated (for example, a prompt "Gordon found 3 optimizations for your Dockerfile. Apply all?"). The challenge for future work is balancing autonomy with safety – perhaps incorporating policies or tests to ensure Gordon's actions do not violate best practices or user intent.

- **Expanded Context Integration:** Another direction is broadening the scope of context that Gordon can use. Docker's **2025 vision** for the agent includes "more context from your registry" and "deeper presence across the development tools you already use" [10]. "Context from your registry" implies that Gordon might integrate with container registries (Docker Hub or private registries) to know, for instance, what images a user/team frequently uses, or to pull in metadata like security scans for those images. This could allow even more tailored advice (e.g., recommending base images that align with the organization's approved images, or detecting that a developer is using an outdated version of an internal image). Deeper presence in dev tools suggests that Gordon might extend beyond Docker Desktop into IDEs or other interfaces. For instance, one could imagine a VS Code extension that surfaces Gordon's suggestions within the code editor when editing a Dockerfile or a Kubernetes manifest. Or integration into CI/CD pipelines – perhaps Gordon could act as a check that comments on pull requests with Dockerfile improvements. These expansions would require new integrations and possibly a modularization of Gordon's components to work outside Docker Desktop. It aligns with a trend of AI copilots being embedded in various stages of the development lifecycle, providing consistency of assistance.

- **Learning from User Interactions:** Currently, Gordon's learning loop is mostly on Docker's side (they gather anonymized feedback and improve the system). A more adaptive future version could personalize its advice based on a user's history or preferences. For example, if a team always uses Alpine-based images, Gordon might prioritize Alpine-flavored suggestions (or conversely, if a user dislikes Alpine, Gordon could stop suggesting it). This kind of personalization would edge towards on-device learning or at least profile-based adjustments, which is complex but potentially rewarding. Academic research could explore how to safely incorporate feedback to tune the assistant's behavior per user without compromising privacy. One could also imagine Gordon integrating with version control to see if its suggestions were adopted or not, learning indirectly from that (though this raises significant technical and privacy questions).

- **Incorporating New AI Models:** The AI field is rapidly evolving. Future iterations of Gordon might leverage more advanced LLMs or specialized models. For example, OpenAI might release new versions with larger context windows or better coding knowledge, which Gordon could immediately benefit from. Alternatively, Docker could experiment with open-source LLMs (possibly running via Docker Model Runner on the user's machine for those who want a local model). If models like Meta's LLaMA or others approach GPT-4 levels, Docker might provide an option for an "offline mode" of Gordon using a local model to address privacy/air-gap use cases. Another avenue is multimodal

models – though less directly relevant, one could imagine a model that could analyze a Docker image or diagram. While not immediate, if a future model could, say, look at a container's filesystem snapshot or an architecture diagram and give advice, that would open new frontiers for the assistant.

- **Enterprise and Team Features:** Docker has hinted at features for enterprise teams such as **customization** and **collaboration** [72]. Future work likely includes allowing organizations to customize Gordon's knowledge base. For example, a company might plug in its internal documentation or policies so that Gordon can answer questions about internal developer workflows or enforce company-specific best practices in its suggestions. This would transform Gordon into a more powerful *team assistant*. Collaboration features might involve Gordon facilitating knowledge sharing – perhaps summarizing how an issue was resolved and suggesting adding it to documentation, or integration with chat platforms (imagine asking Gordon a Docker question in Slack). There is also mention of enhancing security via AI [72], which could mean integrating Gordon with security tools beyond Scout, such as monitoring compliance over time or educating developers about security as they write container configs.

- **User Interface Evolution:** On the front-end, as Gordon's capabilities grow, its UI may evolve from a simple chat. We might see richer visualizations (for example, a mode where Gordon can present a side-by-side diff of a Dockerfile: original vs. optimized). Or a "wizard" style interface for certain tasks guided by Gordon (e.g., a step-by-step containerization wizard driven by AI). The interplay between proactive suggestions and user queries could be improved – Gordon might someday proactively alert: "I notice this container is OOM-killed frequently; would you like advice?" rather than waiting for the user to ask. Such proactivity has been explored in research like *CodingGenie*, which provided proactive code suggestions to developers with some success [64] [65]. Finding the right balance of proactive help in the Docker context would be an interesting area for user experience research.

- **Evaluation and Benchmarking:** From an academic viewpoint, future work should also include rigorous evaluation of Gordon's impact. This could involve user studies to measure how much time is saved in common tasks with Gordon vs without, or what the learning curve is for new Docker users who use Gordon. Telemetry could be analyzed to see which suggestions are most accepted or which questions are most frequently asked, guiding further training. It might also be useful to benchmark Gordon's answers against other assistants (e.g., ask the same Docker questions to ChatGPT, Copilot, and Gordon, and compare accuracy). Such studies would contribute to the literature on AI-assisted development, quantifying benefits and highlighting areas to improve (for instance, if it turns out Gordon struggles with a certain category of problem consistently).

- **Cross-Domain Expansion:** Although Docker's focus is containers, one can foresee the model of Gordon inspiring similar assistants in adjacent domains. Docker could potentially collaborate or allow Gordon to have knowledge about Kubernetes (beyond basic Docker Swarm or Compose). A fully container-centric AI assistant might eventually handle questions about orchestration (Kubernetes, ECS, etc.) since many Docker users graduate to those. This would require feeding it additional documentation and tools. It's speculative, but as the industry moves, tools seldom exist in isolation; maybe Gordon could become a more general "Cloud-Native DevOps Assistant". That said, doing too much could dilute its expertise, so this would be approached carefully.

In conclusion, the future work for Gordon involves both deepening and widening its capabilities: deepening, by allowing more complex and autonomous interactions in the Docker domain; widening, by integrating with more context sources, user customizations, and possibly broader topics. Achieving these will require advances in the underlying AI (especially for trustworthiness and controllability), thoughtful UX design, and continuous feedback from the developer community. Docker has positioned Gordon as an evolving project – indeed, they treat it as part of a "Docker AI" initiative with a roadmap into 2025 and beyond [10]. As researchers and practitioners, watching how Gordon matures will yield valuable insights into how AI agents can be productively woven into everyday developer workflows, enhancing human capabilities while respecting human oversight. Gordon's journey will likely serve as a case study in the successes and hurdles of deploying AI in domain-specific, high-stakes environments like software development, providing lessons that extend to many other domains where AI assistants are making inroads.

---

[1] [3] [9] [14] [16] [17] [18] [19] [21] [22] [24] [27] [30] [32] [33] [34] [44] [45] [48] [62] Meet Gordon: An AI Agent for Docker | Docker
https://www.docker.com/blog/meet-gordon-an-ai-agent-for-docker/

[2] [8] [25] [39] [40] [41] [42] [43] [46] [47] [50] [63] [68] [69] Ask Gordon | Docker Docs
https://docs.docker.com/ai/gordon/

[4] [10] [11] [20] [28] [29] [49] [51] [52] [54] [55] [66] [70] [72] Introducing Beta Launch of Docker AI Agent | Docker
https://www.docker.com/blog/beta-launch-docker-ai-agent/

[5] [6] [37] [38] [57] [58] Amazon Q and Claude Code Let AI Control the Developer CLI - InfoQ
https://www.infoq.com/news/2025/04/amazon-q-cli-claude-code/

[7] [53] [61] Docker Documentation Gets an AI-powered Assistant | Docker
https://www.docker.com/blog/docker-documentation-ai-powered-assistant/

[12] [13] [15] [26] [67] Supercharging Your Docker Workflow with Ask Gordon, Docker's AI Assistant - DEV Community
https://dev.to/prasadb89/supercharging-your-docker-workflow-with-ask-gordon-dockers-ai-assistant-11hp

[23] [35] [36] Function calling and other API updates | OpenAI
https://openai.com/index/function-calling-and-other-api-updates/

[31] [64] [65] CodingGenie: A Proactive LLM-Powered Programming Assistant
https://arxiv.org/html/2503.14724v1

[56] AI Code Assistant – Amazon Q Developer: Build - AWS
https://aws.amazon.com/q/developer/build/

[59] CLI AI Assistant - JFrog Applications
https://docs.jfrog-applications.jfrog.io/jfrog-applications/jfrog-cli/cli-ai

[60] AI Assistant Features - JetBrains
https://www.jetbrains.com/ai-assistant/

[71] Docker Desktop 4.38.0 (181591) doesn't started. #14611 - GitHub
https://github.com/docker/for-win/issues/14611